

DBMS Revision

Nithilan Ramesh

November 14, 2025

Contents

1	Module 1: Introduction & Architecture (Unit 1)	2
1.1	DBMS vs. File Processing Systems	2
1.2	Data Abstraction (3-Schema Architecture)	2
1.3	Data Independence	2
1.4	DBMS Architecture	2
1.5	Database Users	3
2	Module 2: Conceptual Modeling (ER & UML) (Unit 1)	3
2.1	ER Components	3
2.2	Relationships & Constraints	4
2.3	Extended ER Features	4
2.4	UML (Unified Modeling Language)	4
3	Module 3: Relational Model & Algebra (Unit 1)	4
3.1	Relational Algebra (Unary Operators)	4
3.2	Relational Algebra (Binary Operators)	5
3.3	The Joins Family	5
3.4	Mapping ER to Relational Tables	5
4	Module 4: Database Design & Normalization (Unit 2)	5
4.1	Functional Dependencies (FD)	6
4.2	Anomalies	6
4.3	Normal Forms (The Ladder)	6
4.4	Decomposition Algorithms	6
5	Module 5: SQL (Unit 2)	6
5.1	DDL (Data Definition Language)	7
5.2	DML (Data Manipulation Language)	7
5.3	DCL/TCL (Data Control / Transaction Control Language)	7
5.4	Advanced SQL	7
6	Module 6: Storage & Indexing (Unit 3)	7
6.1	File Organization	8
6.2	Indexing Types	8
6.3	B+ Trees	8
6.4	Hashing	8
7	Module 7: Transactions (Unit 3)	8
7.1	ACID Properties	9
7.2	State Transition Diagram	9
7.3	Schedules	9
7.4	Serializability	9

7.5 Recoverability	10
8 Module 8: Concurrency Control (Unit 3)	10
8.1 Locks	10
8.2 2PL (Two-Phase Locking)	10
8.3 Timestamp Ordering	10
8.4 Deadlocks	11

1 Module 1: Introduction & Architecture (Unit 1)

The foundation. Don't just know definitions; know the "Why" and "How".

1.1 DBMS vs. File Processing Systems

- **File System Drawbacks:**
 - **Data Redundancy & Inconsistency:** Same data duplicated in multiple files, leading to conflicts if not all copies are updated.
 - **Difficulty in Accessing Data:** Requires writing a new program for every new query.
 - **Data Isolation:** Data is scattered in various files with different formats.
 - **Integrity Problems:** Constraints (e.g., account balance > 0) are hard to enforce.
 - **Atomicity Problems:** A transaction (e.g., fund transfer) might fail halfway, leaving the system in an inconsistent state.
 - **Concurrent Access Anomalies:** Uncontrolled concurrent access can lead to inconsistencies.
 - **Security Problems:** Hard to enforce access restrictions on a per-user basis.

1.2 Data Abstraction (3-Schema Architecture)

- **Purpose:** To hide the complexity of data storage from users.
- **Physical Level:** The lowest level. Describes **how** the data is actually stored (e.g., on disk in blocks, files, data structures).
- **Logical Level:** The middle level. Describes **what** data is stored and the relationships between them (e.g., tables, columns, keys). This is the DBA's view.
- **View Level:** The highest level. Describes a part of the database for a particular user group. Hides details and provides a level of security.

1.3 Data Independence

- **Physical Data Independence:** The ability to modify the physical schema without changing the logical schema. (e.g., changing storage device, file organization, or indexing). The logical level is unaffected.
- **Logical Data Independence:** The ability to modify the logical schema without changing the external schemas or application programs. (e.g., adding a new column to a table). Harder to achieve than physical independence.

1.4 DBMS Architecture

- **Query Processor:**
 - **DDL Interpreter:** Interprets DDL statements (e.g., 'CREATE TABLE') and records them in the data dictionary.

- **DML Compiler:** Translates DML statements (e.g., ‘SELECT’) into low-level instructions that the Query Evaluation Engine understands. Involves query optimization.
- **Query Evaluation Engine:** Executes the low-level instructions generated by the DML compiler.
- **Storage Manager:**
 - **Authorization & Integrity Manager:** Checks for user authorization and enforces integrity constraints.
 - **Transaction Manager:** Ensures that the database remains in a consistent state despite system failures and concurrent transaction execution (ACID properties).
 - **File Manager:** Manages the allocation of space on disk storage and the data structures used to represent information.
 - **Buffer Manager:** Manages the transfer of data between main memory and secondary storage. Caches data in memory for faster access.

1.5 Database Users

- **Naive Users:** Interact with the system through a predefined user interface (e.g., a bank teller using a banking application).
- **Application Programmers:** Professionals who write application programs that interact with the database.
- **Sophisticated Users:** Interact with the system without writing programs, using query languages like SQL (e.g., data analysts).
- **Specialized Users:** Write specialized database applications that do not fit into the traditional data-processing framework (e.g., CAD, expert systems).
- **Database Administrator (DBA):** The central authority. Responsibilities include schema definition, storage structure, access authorization, and routine maintenance.

2 Module 2: Conceptual Modeling (ER & UML) (Unit 1)

Converting real-world requirements into diagrams.

2.1 ER Components

- **Entities:** A real-world object. Represented by a rectangle.
 - **Strong Entity:** Has its own primary key. (e.g., ‘Student’).
 - **Weak Entity:** Dependent on a strong entity for its existence and does not have its own primary key. Has a partial key (discriminator). Represented by a double rectangle. (e.g., ‘Dependent’ of an ‘Employee’).
- **Attributes:** Properties of an entity. Represented by an ellipse.
 - **Simple:** Cannot be subdivided (e.g., ‘Age’).
 - **Composite:** Can be subdivided (e.g., ‘Address’ -> ‘Street’, ‘City’, ‘Zip’).
 - **Multivalued:** Can have more than one value (e.g., ‘PhoneNumber’). Represented by a double ellipse.
 - **Derived:** Value can be calculated from other attributes (e.g., ‘Age’ from ‘DateOfBirth’). Represented by a dashed ellipse.
 - **Key Attribute:** Uniquely identifies an entity. Underlined.

2.2 Relationships & Constraints

- **Relationship:** An association among several entities. Represented by a diamond.
- **Cardinality:** The number of instances of one entity that can be associated with an instance of another entity.
 - **1:1:** One to One (e.g., A ‘Manager’ heads one ‘Department’).
 - **1:M:** One to Many (e.g., A ‘Professor’ teaches many ‘Courses’).
 - **M:N:** Many to Many (e.g., ‘Students’ enroll in many ‘Courses’).
- **Participation:** Specifies whether an entity’s existence depends on its relationship to another entity.
 - **Total Participation:** Every entity instance must participate in the relationship. Represented by a double line from the entity to the relationship. (e.g., Every ‘Loan’ must belong to a ‘Customer’).
 - **Partial Participation:** Not all entity instances must participate. Represented by a single line. (e.g., Not every ‘Customer’ has to have a ‘Loan’).

2.3 Extended ER Features

- **Specialization:** Top-down design process. Defining a set of subclasses of an entity type. (e.g., ‘Person’ can be specialized into ‘Student’ and ‘Employee’). An “IS-A” relationship.
- **Generalization:** Bottom-up design process. Combining a number of entity sets that share the same features into a higher-level entity set. (e.g., ‘Car’ and ‘Truck’ can be generalized into ‘Vehicle’).
- **Constraints on Specialization/Generalization:**
 - **Disjoint:** An entity can be a member of at most one of the subclasses.
 - **Overlapping:** An entity can be a member of more than one subclass.
 - **Total:** An entity must belong to one of the subclasses.
 - **Partial:** An entity is not required to belong to any of the subclasses.
- **Aggregation:** Allows a relationship set to be treated as a higher-level entity set for the purpose of another relationship. Used to model relationships between relationships.

2.4 UML (Unified Modeling Language)

- A standard for software modeling, not specific to databases but can be used for conceptual design.
- **Class Diagrams:** Very similar to ER diagrams. Show classes (entities), their attributes, methods, and relationships between classes.
- **Use Case Diagrams:** Show interactions between users (actors) and the system. Good for capturing functional requirements.
- **Activity Diagrams:** Flowchart-like diagrams that show the flow of control from one activity to another.

3 Module 3: Relational Model & Algebra (Unit 1)

The mathematical engine behind SQL.

3.1 Relational Algebra (Unary Operators)

- σ (**Select**): Filters rows based on a condition. ‘ $\sigma_{\text{salary} > 50000}(\text{Employee})$ ’
- π (**Project**): Selects a subset of columns (attributes). ‘ $\pi_{\text{name, salary}}(\text{Employee})$ ’
- ρ (**Rename**): Renames a relation or its attributes. ‘ $\rho_E(\text{Employee})$ ’ or ‘ $\rho_{\text{Emp(N, S)}}(\text{Employee})$ ’

3.2 Relational Algebra (Binary Operators)

- **Set Operations:**
 - **Union ($\cup$):** Combines rows from two relations, removes duplicates.
 - **Intersection ($\cap$):** Returns rows that exist in both relations.
 - **Set Difference ($-$):** Returns rows that are in the first relation but not in the second.
 - **Note:** Relations must be **Union Compatible** (same number of attributes, and corresponding attributes have the same domain).
- **Cartesian Product ($\times$):** Combines every row from the first relation with every row from the second. ' $R \times S$ '.
- **Division ($\div$):** Used for "for all" queries. ' $A \div B$ ' finds tuples in A that are associated with **all** tuples in B. (e.g., Find students who have taken all required courses).

3.3 The Joins Family

- **Inner Joins:** Combine and keep only the rows that match the join condition.
 - **Theta Join:** A join based on any comparison operator ('<', '>', '=', etc.). ' $R \bowtie_{\text{condition}} S$ '.
 - **Equi Join:** A theta join where the condition is only equality.
 - **Natural Join ($\bowtie$):** *Anequi join on all common attributes. The duplicate common attribute is removed from the result.*
- **Outer Joins:** Keep all rows from one or both relations, even if there is no match. Fills missing values with NULL.
 - **Left Outer Join ($\leftarrow$):** Keeps all rows from the left relation.
 - **Right Outer Join ($\rightarrow$):** Keeps all rows from the right relation.
 - **Full Outer Join ($\ltimes$):** Keeps all rows from both relations.

3.4 Mapping ER to Relational Tables

- **Rule 1 (Strong Entity):** Create a table for the entity. The entity's attributes become columns. The primary key of the entity becomes the primary key of the table.
- **Rule 2 (Weak Entity):** Create a table for the weak entity. Include all its attributes. Add the primary key of the owner entity as a foreign key. The primary key of this table is the combination of the owner's PK and the weak entity's partial key.
- **Rule 3 (1:M Relationship):** Take the primary key from the '1' side and add it as a foreign key to the 'M' side table.
- **Rule 4 (M:N Relationship):** Create a new table (junction/linking table). Its primary key is a composite of the primary keys from the two participating entities.
- **Rule 5 (Multivalued Attribute):** Create a new table for the attribute. This table will have the primary key of the entity (as a foreign key) and a column for the multivalued attribute. The combination of these two columns forms the primary key for this new table.

4 Module 4: Database Design & Normalization (Unit 2)

Structuring tables to avoid redundancy. (High Exam Weight)

4.1 Functional Dependencies (FD)

- **Definition:** A constraint between two sets of attributes. For a relation R, ' $X \rightarrow Y$ ' means that for any two tuples $t1$ and $t2$ in R, if ' $t1[X] = t2[X]$ ', then ' $t1[Y] = t2[Y]$ '. The value of X determines the value of Y.
- **Armstrong's Axioms:**
 - **Reflexivity:** If Y is a subset of X, then $X \rightarrow Y$. (Trivial)
 - **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$.
 - **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.
- **Attribute Closure (X^+):** The set of all attributes that are functionally determined by X. Used to find candidate keys and check dependencies.

4.2 Anomalies

- **Insertion Anomaly:** Cannot insert a fact about one entity until we have a fact about another. (e.g., Cannot add a new course unless a student is enrolled in it).
- **Deletion Anomaly:** Deleting a tuple may cause the loss of unrelated information. (e.g., Deleting the last student from a course might delete the course itself).
- **Update Anomaly:** A value must be updated in multiple places, which can lead to inconsistency if not all are updated. (e.g., A course name is stored with every student enrollment record).

4.3 Normal Forms (The Ladder)

- **1NF (First Normal Form):** All attributes must be atomic (indivisible). No multivalued attributes or composite attributes. The table must have a primary key.
- **2NF (Second Normal Form):** Must be in 1NF. No partial dependencies. A non-prime attribute (not part of any candidate key) must not depend on a part of a composite candidate key. (Only relevant for tables with composite keys).
- **3NF (Third Normal Form):** Must be in 2NF. No transitive dependencies. A non-prime attribute must not depend on another non-prime attribute. (i.e., for ' $X \rightarrow A$ ', either X is a superkey or A is a prime attribute).
- **BCNF (Boyce-Codd Normal Form):** A stricter version of 3NF. For every non-trivial FD ' $X \rightarrow Y$ ', X must be a superkey. It eliminates redundancies that 3NF might miss.

4.4 Decomposition Algorithms

- **Lossless Join Property:** When a relation R is decomposed into R1 and R2, the join of R1 and R2 must yield the original relation R without creating spurious (extra) tuples. A decomposition $\{R1, R2\}$ is lossless if ' $(R1 \cap R2) \rightarrow R1$ ' or ' $(R1 \cap R2) \rightarrow R2$ '.
- **Dependency Preservation:** When R is decomposed, all the original FDs must still be enforceable by checking constraints on the new, smaller relations. It should not be necessary to join the relations back to check an FD.

5 Module 5: SQL (Unit 2)

The practical language.

5.1 DDL (Data Definition Language)

- Defines the database schema.
- **CREATE**: To create databases, tables, views, indexes. ‘CREATE TABLE Students (...)’
- **ALTER**: To modify the structure of a database object. ‘ALTER TABLE Students ADD COLUMN gpa DECIMAL(3,2);’
- **DROP**: To delete database objects. ‘DROP TABLE Students;’
- **TRUNCATE**: To remove all records from a table, but not the table itself. Faster than DELETE.

5.2 DML (Data Manipulation Language)

- Used for accessing and manipulating data.
- **SELECT**: To retrieve data from a database. ‘SELECT name, major FROM Students WHERE id = 101;’
- **INSERT**: To insert new records into a table. ‘INSERT INTO Students VALUES (...);’
- **UPDATE**: To modify existing records. ‘UPDATE Students SET major = ‘CS’ WHERE id = 101;’
- **DELETE**: To delete records. ‘DELETE FROM Students WHERE id = 101;’

5.3 DCL/TCL (Data Control / Transaction Control Language)

- **GRANT**: To give user access privileges to the database. ‘GRANT SELECT ON Students TO user1;’
- **REVOKE**: To withdraw user access privileges. ‘REVOKE SELECT ON Students FROM user1;’
- **COMMIT**: To save all the work done in a transaction.
- **ROLLBACK**: To undo all the work done since the last COMMIT.

5.4 Advanced SQL

- **Nested Subqueries**: A query within another query. Can be in the ‘WHERE’, ‘FROM’, or ‘SELECT’ clause.
 - ‘SELECT name FROM Students WHERE major_id IN (SELECT id FROM Majors WHERE name = ‘Computer Science’);’
- **Aggregate Functions**: Perform a calculation on a set of values and return a single value.
 - ‘COUNT()’, ‘SUM()’, ‘AVG()’, ‘MAX()’, ‘MIN()’.
- **GROUP BY**: Groups rows that have the same values in specified columns into summary rows. Often used with aggregate functions.
 - ‘SELECT major, AVG(gpa) FROM Students GROUP BY major;’
- **HAVING**: Filters groups based on a condition, after ‘GROUP BY’ has been applied. ‘WHERE’ filters rows before grouping.
 - ‘SELECT major, AVG(gpa) FROM Students GROUP BY major HAVING AVG(gpa) > 3.5;’

6 Module 6: Storage & Indexing (Unit 3)

How data sits on the disk.

6.1 File Organization

- **Sequential (Ordered):** Records are stored in order based on the value of a key field. Fast for range queries but slow for insertion/deletion.
- **Heap (Unordered):** Records are placed in the file in the order they are inserted. Insertion is very fast, but searching requires a linear scan.
- **Clustered:** Related records from different tables are stored together in the same disk block. Improves join performance but can only be done on one set of relationships.

6.2 Indexing Types

- **Primary Index:** Index on a key field that is used to order the file (the search key).
- **Secondary Index:** An index whose search key specifies an order different from the sequential order of the file. Can be on a candidate key or a non-key field.
- **Dense Index:** An index entry appears for every search-key value in the file.
- **Sparse Index:** An index entry appears for only some of the search-key values (e.g., for the first record in each block). Requires less space but is slower for lookups.

6.3 B+ Trees

- A balanced tree structure that is the de-facto standard for indexing in most DBMS.
- **Structure:**
 - **Internal Nodes:** Contain only key values and pointers to other nodes.
 - **Leaf Nodes:** Contain key values and pointers to the actual data records.
 - All leaf nodes are at the same level.
 - Leaf nodes are linked together in a doubly-linked list.
- **Why it is best for Range Queries:** The linked list at the leaf level allows for efficient traversal of records in sorted order. ‘SELECT * FROM Employee WHERE age BETWEEN 30 AND 40;’

6.4 Hashing

- Maps key values directly to a storage location (bucket).
- **Static Hashing:** The number of buckets is fixed.
 - **Problem:** Performance degrades significantly with overflow (when a bucket is full). Requires periodic reorganization.
- **Dynamic (Extendible) Hashing:** The hash function and number of buckets can be changed dynamically as the database grows or shrinks. Avoids major reorganization.

7 Module 7: Transactions (Unit 3)

Ensuring integrity.

7.1 ACID Properties

- **Atomicity:** A transaction is an “all or nothing” unit. It either fully completes, or it is rolled back as if it never happened.
- **Consistency:** A transaction brings the database from one valid state to another. All integrity constraints must be satisfied.
- **Isolation:** Concurrent transactions should not interfere with each other. The result of concurrent execution should be the same as if they were executed serially.
- **Durability:** Once a transaction has been committed, its effects are permanent and will survive any subsequent system failure (e.g., power loss, crash).

7.2 State Transition Diagram

- **Active:** The initial state; the transaction is executing.
- **Partially Committed:** After the final statement has been executed.
- **Committed:** After successful completion.
- **Failed:** After the discovery that normal execution can no longer proceed.
- **Aborted:** After the transaction has been rolled back and the database restored to its state prior to the start of the transaction.

7.3 Schedules

- A sequence of operations (read, write, abort, commit) from a set of concurrent transactions.
- **Serial Schedule:** Operations of different transactions are not interleaved. They are executed one after another. Always consistent.
- **Concurrent (Non-serial) Schedule:** Operations of different transactions are interleaved. Improves throughput but can lead to inconsistency if not managed.

7.4 Serializability

- A concurrent schedule is serializable if it is equivalent to some serial schedule. This is the benchmark for correctness.
- **Conflict Serializability:** A schedule is conflict serializable if it can be transformed into a serial schedule by a series of non-conflicting swaps of adjacent operations.
 - **Test:** Draw a **Precedence Graph**. A node for each transaction. An edge from T_i to T_j if T_i performs an operation that conflicts with one in T_j , and T_i 's operation occurs first. The schedule is conflict serializable if and only if the graph has **no cycles**.
- **View Serializability:** A more relaxed form. A schedule is view serializable if it is view equivalent to some serial schedule. Allows for some schedules that are not conflict serializable but are still correct. (Harder to check).

7.5 Recoverability

- **Recoverable Schedule:** A schedule where, for any pair of transactions T_i and T_j , if T_j reads a data item previously written by T_i , the commit operation of T_i must appear before the commit operation of T_j .
- **Cascadeless Schedule:** A stricter schedule where, for every pair of transactions T_i and T_j , if T_j reads a data item previously written by T_i , the commit operation of T_i must appear before the read operation of T_j . This avoids **cascading rollbacks** (if T_i aborts, T_j must also be rolled back). No “dirty reads”.
- **Strict Schedule:** The strictest form. A transaction can neither read nor write a data item 'X' until the last transaction that wrote 'X' has committed or aborted.

8 Module 8: Concurrency Control (Unit 3)

Managing traffic.

8.1 Locks

- A mechanism to control access to data items.
- **Binary Locks:** A lock has two states: locked (1) or unlocked (0). Very simple but not flexible.
- **Shared (S) / Exclusive (X) Locks:**
 - **Shared (Read) Lock:** Multiple transactions can hold an S lock on an item simultaneously. ‘S-lock’ is requested for reading.
 - **Exclusive (Write) Lock:** Only one transaction can hold an X lock on an item at any time. No other lock (S or X) can be held. ‘X-lock’ is requested for writing.

8.2 2PL (Two-Phase Locking)

- A protocol that ensures serializability. It has two phases for each transaction:
- **Growing Phase:** The transaction can acquire locks but cannot release any locks.
- **Shrinking Phase:** The transaction can release locks but cannot acquire any new locks.
- **Strict 2PL:** A variation of 2PL where all **exclusive** locks held by a transaction are released only after the transaction commits or aborts. Avoids cascading rollbacks.
- **Conservative (Static) 2PL:** Requires a transaction to lock all the data items it needs in advance. If any item cannot be locked, the transaction waits and releases any locks it already acquired. Deadlock-free but impractical.

8.3 Timestamp Ordering

- An alternative to locking. Each transaction is assigned a unique, increasing timestamp. The order of execution is determined by the timestamp order.
- If a transaction T_i tries to access data that has been accessed by a younger (later) transaction T_j , T_i is aborted and restarted with a new timestamp.
- **Wait-Die:** If T_i (older) requests a resource held by T_j (younger), T_i waits. If T_j (younger) requests a resource held by T_i (older), T_j dies (is rolled back). Non-preemptive.
- **Wound-Wait:** If T_i (older) requests a resource held by T_j (younger), T_i wounds T_j (forces rollback). If T_j (younger) requests a resource held by T_i (older), T_j waits. Preemptive.

8.4 Deadlocks

- A situation where two or more transactions are waiting indefinitely for each other to release locks.
- **Detection:** Use a **Wait-for Graph**. A node for each transaction. An edge from T_i to T_j if T_i is waiting for a resource held by T_j . A cycle in the graph indicates a deadlock.
- **Prevention:**
 - **Timestamp schemes** like Wait-Die and Wound-Wait are deadlock-free.
 - **Conservative 2PL** is deadlock-free.
- **Recovery:**
 - **Victim Selection:** Choose a transaction to abort to break the cycle. Factors include how long the transaction has run, how much data it has used, etc.
 - **Rollback:** The victim transaction is rolled back, releasing its resources and allowing other transactions to proceed.